

Lab 4 - CRUD forma, rich JS, dropdown

Predaja: 22.5.

Zadaci i bodovanje

Kriterij	Bodovi
Kreiranje kompletno funkcionalne CRUD podrške za sve entitete	2
Kreiranje padajućeg izbornika s AJAX autocomplete opcijom pretrage	2
Implementacija validacije (client side + server side)	1
Napredno korištenje JavaScripta	1
Datumska kontrola (partial view)	1

Nužni uvjeti za predaju vježbe

- ☐ Potpuno funkcionalne stranice za pregled, pretragu, unos, uređivanje i brisanje entiteta gdje poslovna pravila dopuštaju
 - ☐ Svaka stranica koja prikazuje listu podataka treba imati AJAX pretragu
 - ☐ Ako neki CRUD endpoint ne radi ispravno, oduzimaju se bodovi
- ☐ Dropdown s autocomplete opcijom
 - ☐ Napraviti custom kontrolu koja omogućuje pretragu povezanih podataka, primjerice gradova ili korisnika, a ponaša se kao dropdown
 - ☐ Autocomplete mora koristiti AJAX za asinkrono dohvaćanje rezultata sa servera
- ☐ Validacija - client side + server side
 - ☐ Client side validacija - kad kontrola izgubi fokus, validacija se mora "okinuti"
 - ☐ Server side validacija - uvijek mora postojati validacija na serverskoj strani
 - ☐ Validacijske poruke moraju se lijepo uklapati u sučelje

- ☐ Napraviti animacije koje su u službi aplikacije i koje ilustriraju napredno korištenje JavaScripta
- ☐ Napraviti datumsku kontrolu (datum+vrijeme)
 - ☐ Napraviti preko partial view
 - ☐ Primjeniti na svim mjestima gdje se koristi datum
 - ☐ Osigurati da radi na hr+en formatu ovisno o postavkama preglednika
 - ☐ NE koristiti default datepicker kontrolu iz browsera - mora biti ili JS plugin ili kompletno kodirano

CRUD operacije u kontekstu Entity Frameworka

U sklopu ove vježbe potrebno je za entitete podržati osnovne CRUD operacije: kreiranje, uređivanje i brisanje zapisa. Iako se na razini korisničkog sučelja ove operacije čine sličnima, u pozadini imaju različite zahtjeve.

Delete

`Delete` je najjednostavnija operacija jer u pravilu prima samo `ID` zapisa koji se briše. Najjednostavnija implementacija dohvaća entitet preko ID-a i poziva `Remove` nad `DbSet` kolekcijom.

Kod brisanja je potrebno razmisliti o relacijama između podataka. Ako entitet ima povezane zapise, treba znati hoće li se koristiti:

- `cascade delete` — brišu se i povezani zapisi
- `set null` — strani ključ se postavlja na `null`
- zabrana brisanja — operacija ne uspijeva ako postoje povezani podaci

Soft delete

U stvarnim aplikacijama često je bolje ne brisati zapis fizički iz baze, nego koristiti **soft delete**. To znači da entitet dobije polje poput `DeletedAt`, koje je `null` dok je zapis aktivan, a popunjava se datumom brisanja kada korisnik obriše zapis.

Prednost soft delete pristupa je mogućnost lakšeg oporavka podataka. Nedostatak je što svaka lista, pretraga i dohvat podataka moraju paziti da ne prikazuju obrisane zapise.

Primjer pravila:

- kod brisanja ne pozivati `Remove`, nego postaviti `DeletedAt = DateTime.UtcNow`
- kod dohvaćanja liste uvijek filtrirati zapise gdje je `DeletedAt == null`

Create operacija

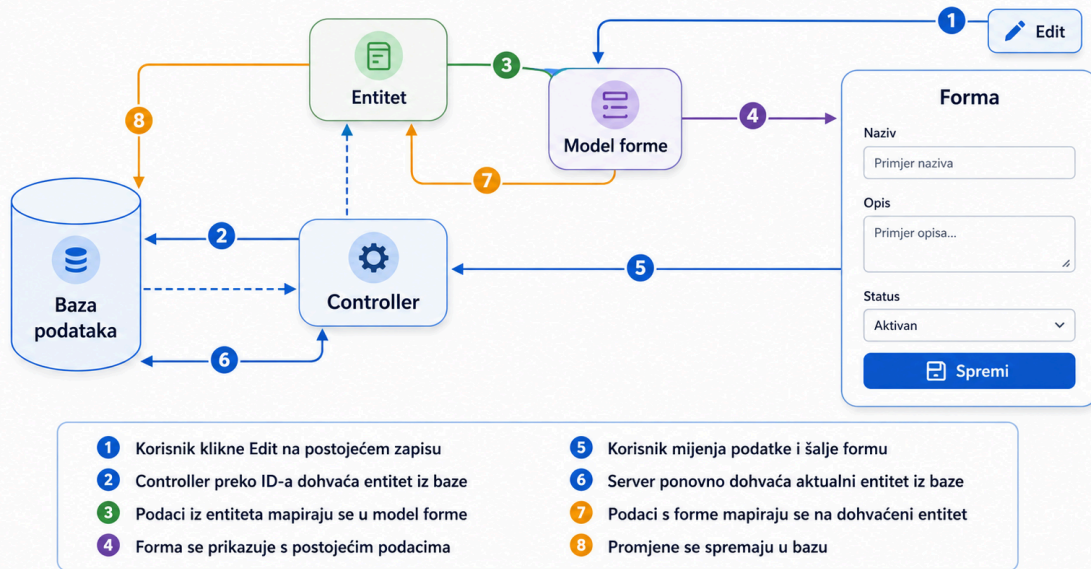
Kod `Create` operacije controller prima podatke potrebne za stvaranje novog zapisa. Ti podaci najčešće dolaze s forme, ali mogu doći i iz neke druge akcije, primjerice gumba koji automatski kreira zapis s unaprijed zadanim vrijednostima.



Edit operacija

`Edit` je složeniji od `Create` operacije jer se radi s postojećim zapisom.

Tipičan tijek uređivanja postojećeg zapisa



Entiteti vs modeli forme

Iako je tehnički moguće direktno povezati podatke s forme na entitet, u pravilu je bolje koristiti posebne klase za forme, primjerice:

- `QuizCreateModel`
- `QuizEditModel`
- `ClientCreateModel`
- `ClientEditModel`

Razlozi:

- entitet može imati polja koja se ne smiju slati na frontend
- entitet može imati interna polja koja korisnik ne uređuje
- forma može imati polja koja ne odgovaraju 1:1 strukturi baze
- sigurnije je eksplicitno kontrolirati što korisnik može mijenjati

Primjer problema je korisnički entitet. U bazi može sadržavati polja poput `PasswordHash`, `PasswordSalt`, `IsLocked`, `LockoutEndTime` ili `LastLoginAt`. Takva polja ne želimo slati na formu niti dopustiti da ih korisnik mijenja.

Zato je bolji pristup:

1. dohvatiti entitet iz baze
2. mapirati potrebna polja u model forme

3. prikazati model forme korisniku
4. kod spremanja ponovno dohvatiti entitet
5. mapirati samo dopuštena polja iz modela forme natrag u entitet

Danas je ovaj pristup praktičniji jer AI alati mogu brzo generirati mapiranje između entiteta i modela forme. I dalje je potrebno provjeriti rezultat, ali ručni posao više nije toliko velik.

Metoda `TryUpdateModel`

Do sada smo spominjali u kontekstu povezivanja modela s podacima s forme jedino mogućnost automatskog povezivanja na način da se direktno podaci s forme spremaju u model (akcija `Create`).

Međutim, takav način nam ne odgovara uvijek, pogotovo kod uređivanja podataka iz baze. Podsjetimo se kako izgleda proces kompletnog prikaza edit forme i spremanja podataka natrag u bazu korištenjem repository obrasca:

1. Poziva se akcija `Edit`.
 - a. Dohvaća se podatak iz baze, primjerice kompanija s konkretnom `ID` vrijednošću.
 - b. Prikazuje se forma s već popunjenim podacima iz baze, odnosno model `Client`.
2. Kod slanja forme na server poziva se akcija `Edit` (`HttpPost`) koja kao parametar prima model `Client`.
 - a. Spremamo tog klijenta u bazu podataka.

Koji je problem u gornjem scenariju? Što ako klijent ima neka polja koja se ne prikazuju na formi? Što će biti s tim podacima?

Zbog gornjeg problema sigurnije je koristiti funkciju `TryUpdateModel`, i to na sljedeći način:

1. Poziva se akcija `Edit`.
 - a. Dohvaća se podatak iz baze, primjerice klijent s konkretnom `ID` vrijednošću.
 - b. Prikazuje se forma s već popunjenim podacima iz baze, odnosno model `Client`.

2. Kod slanja forme na server poziva se akcija `Edit` (`HttpPost`) koja kao parametar prima model `Client`.
 - a. Dohvaća se trenutno aktualni klijent iz baze.
 - b. Nad dohvaćenim klijentom poziva se funkcija `TryUpdateModel`, koja podatke s forme zapiše u model.
 - c. Spremamo tog klijenta u bazu podataka.

Najoptimalnije bi bilo koristiti potpuno drugi objekt kao model koji se šalje u view, primjerice `ClientViewModel`, i zatim premapirati pojedina polja u entitet `Client`. Takva arhitektura također zna biti pomalo problematična jer je za većinu entiteta iz baze dovoljan i gore opisan scenarij.

Dobar video o korištenju funkcije `UpdateModel` iz starije verzije ASP.NET MVC-a, ali i dalje primjenjivo, kao i atributa `ActionName`:

<https://www.youtube.com/watch?v=uXwmyuvrn1E>

Problem zastarjelih podataka

Kod edit forme postoji problem ako korisnik dugo drži formu otvorenu. Primjerice, korisnik otvori formu, ode na pauzu i tek nakon 20 minuta klikne `Save`. U međuvremenu je drugi korisnik mogao promijeniti isti zapis.

Ako aplikacija samo spremi podatke iz stare forme, može pregaziti tuđe promjene.

Jedno moguće rješenje je korištenje verzije ili timestampa:

1. Kod otvaranja forme pošalje se i informacija o verziji zapisa.
2. Kod spremanja server uspoređuje verziju iz forme s aktualnom verzijom u bazi.
3. Ako se verzije razlikuju, spremanje se odbija.
4. Korisniku se prikaže poruka da su podaci u međuvremenu promijenjeni i da treba ponovno učitati formu.

Za ovu vježbu nije nužno implementirati taj mehanizam, ali je važno razumjeti problem jer se pojavljuje u aplikacijama s više korisnika koji istovremeno uređuju iste podatke.

Atribut `ActionName`

Kod izrade akcije za uređivanje ili kreiranje novih objekata nailazimo na problem ako želimo imati dvije ovakve akcije, primjerice za `Edit` akciju:

`ClientController.cs`

```
public ActionResult Edit(int id)
{
    ...
}

[HttpPost]
public ActionResult Edit(int id)
{
    this.UpdateModel(...);
    ...
}
```

Ove dvije funkcije su identične po potpisu i ne mogu se prevesti u C# jeziku.

Kako bismo ipak zadržali svojstvo da URL tih akcija bude identičan, odnosno da se razlikuje samo po metodi `GET` ili `POST`, možemo koristiti sljedeći pristup:

`ClientController.cs`

```
[ActionName("Edit")]
public ActionResult EditGet(int id)
{
    ...
}

[HttpPost]
[ActionName("Edit")]
public ActionResult EditPost(int id)
{
    this.UpdateModel(...);
    ...
}
```

Validacija

Validacija je jedan od bitnijih segmenata svake aplikacije. Osnovni koncepti validacije su:

- Onemogućiti spremanje podataka koji nisu konzistentni.
 - Validacija na klijentu, prije slanja na server.
 - Validacija na serveru.
 - Validacija na razini baze podataka.
- Prikazati adekvatnu poruku korisniku u slučaju pogreške.

ASP.NET MVC framework nudi gotova rješenja za validaciju, ali također omogućava naprednu prilagodbu validacijskih mehanizama na svim razinama, odnosno na klijentu i serveru. O tome će biti više govora u kasnijim predavanjima.

Tri razine validacije

Validaciju možemo promatrati na tri razine:

1. Validacija na klijentu
2. Validacija na serveru
3. Validacija u bazi podataka

Sve tri razine imaju različitu svrhu.

Client side validacija

Validacija na klijentu služi prvenstveno za bolji UX. Korisnik odmah vidi da neko polje nije ispravno popunjeno, bez čekanja odgovora servera.

Primjeri:

- obvezno polje nije popunjeno
- email nije u ispravnom formatu
- broj nije u dopuštenom rasponu
- datum nije u očekivanom formatu

Međutim, client side validaciji se ne smije vjerovati. Korisnik je može zaobići slanjem vlastitog HTTP zahtjeva ili izmjenom podataka u browseru.

Server side validacija

Validacija na serveru najvažnija je za sigurnost aplikacije. Server mora provjeriti sve što je važno, čak i ako ista provjera već postoji na klijentu.

Server provjerava:

- jesu li podaci ispravni
- ima li korisnik pravo napraviti akciju
- smije li korisnik vidjeti ili mijenjati konkretan zapis
- krši li zahtjev poslovna pravila aplikacije

Primjer: korisnik možda smije uređivati vlastite podatke, ali ne i podatke drugog korisnika.

Validacija u bazi podataka

Baza podataka služi kao zadnja linija obrane za konzistentnost podataka.

Primjeri:

- obvezni stupci ne smiju biti `null`
- strani ključevi moraju pokazivati na postojeće zapise
- jedinstvena polja ne smiju imati duplikate
- tipovi podataka moraju odgovarati definiciji stupaca

Primjer: ako JMBG mora biti jedinstven, baza bi trebala imati constraint koji sprječava unos duplikata.



Kod korištenja Entity Frameworka standardni LINQ upiti su parametrizirani, pa SQL injection u pravilu nije problem na isti način kao kod ručnog slaganja SQL stringova. I dalje treba izbjegavati ručno spajanje SQL naredbi iz korisničkog unosa.

Anotacije modela

Osnovna metoda validacije je anotacijama pojedinih svojstava u modelu.

Moguće je definirati validaciju i neovisno od modela, no s obzirom na to da se MVC paradigma uvelike temelji na modelima, gotovo uvijek je preporučljivo koristiti jasno definiranu klasu kao model prilikom izrade formi za unos i izmjenu podataka.

Primjena validacije sastoji se od sljedećih koraka:

- Anotirati željena svojstva u modelu.
- U viewu pozvati naredbu za prikaz validacijske poruke, ako se koristi.
- U controlleru provjeriti je li model prošao validaciju.

Validacija obveznog polja (**Required**)

Jedan od najčešćih i najjednostavnijih tipova validacije je obvezno polje. U sljedećem primjeru postaviti ćemo da je obvezno polje pri unosu podataka o kvizovima naziv kviza (**Title**).

Budući da se model klasa prema kojoj prikazujemo view nalazi u projektu `QuizManager.Model`, potrebno je osigurati da taj projekt ima referencu na biblioteku `System.ComponentModel.DataAnnotations`. U .NET Core aplikaciji automatski je definiran skup međuzavisnosti (**Dependencies**) i najvjerojatnije se navedena biblioteka tamo već nalazi.

Nakon toga možemo dodati atribut **Required** na svojstvo u koje spremamo naslov kviza.

`Quiz.cs`

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace QuizManager.Model
{
    public class Quiz
    {
        public int Id { get; set; }

        [Required]
        public string Title { get; set; }

        public string Keywords { get; set; }
        public string Author { get; set; }
    }
}
```

```

        public DateTime DateCreated { get; set; }
        public QuizCategory Category { get; set; }
        public List<Question> Questions { get; set; }
    }
}

```

Budući da pri dodavanju `[Required]` atributa unutar klase `Quiz` nije poznat imenski prostor u kojem se ta klasa nalazi, moguće je ostaviti Visual Studio alatu da ga sam pronade.



Napomena: Od .NET 6 koriste se posebni načini za označavanje polja koja su nullable. Samim time, čak će i `string` svojstva biti not-null, odnosno `Required`, ako nema eksplicitne oznake da su nullable. Primjer: `string? Prezime`.

U idućem koraku potrebno je omogućiti da se klijentu prikaže odgovarajuća poruka ako validacija nije zadovoljena, primjerice ako korisnik ostavi prazno polje za naslov kviza.

`_CreateOrEdit.cshtml`

```

<div class="form-group">
    <label asp-for="Email" class="control-label"></label>
    <input asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>
</div>

```

Potrebno je dodati i odgovarajuće JavaScript biblioteke kako bi klijentska validacija funkcionirala ispravno. U predlošku novog projekta osnovne validacijske skripte dodane su automatski unutar partial viewa

`_ValidationScriptsPartial`.

Dodat ćemo sljedeći odsječak u `_Layout.cshtml` kako bi validacija bila dostupna na svim formama koje radimo u projektu.

`_Layout.cshtml`

```

<footer class="border-top footer text-muted">
    <div class="container">

```

```

&copy; 2021 - Vjezba.Web - <a asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
</div>
</footer>

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<partial name="_ValidationScriptsPartial" />
<script src="~/js/site.js" asp-append-version="true"></script>
@await RenderSectionAsync("Scripts", required: false)
</body>

```

Nakon toga, ako pokrenemo aplikaciju i pokušamo dodati novi kviz preko sučelja, dobit ćemo ispravnu validacijsku pogrešku. Međutim, na taj način se nismo u potpunosti osigurali jer je moguće simulirati zahtjev koji se šalje s podacima forme i na serveru i dalje unositi krive podatke.

Iz tog razloga potrebno je prije obrade informacija na serveru provjeriti prolaze li podaci validaciju.

QuizController.cs

```

[HttpPost]
public IActionResult Create(Quiz model)
{
    if (ModelState.IsValid)
    {
        this._dbContext.Quizes.Add(model);
        this._dbContext.SaveChanges();
        return RedirectToAction(nameof(Index));
    }

    return View(model);
}

[HttpPost, ActionName("Edit")]
public async Task<IActionResult> EditPost(int id)
{

```

```

    var quiz = this._dbContext.Quizes.FirstOrDefault(p =>
p.ID == id);
    var ok = await this.TryUpdateModelAsync(quiz);

    if (ok)
    {
        this._dbContext.SaveChanges();
        return RedirectToAction(nameof(Index));
    }

    return View(quiz);
}

```



Općenito, za sve daljnje zadatke obvezno je koristiti navedenu validaciju za obvezna polja ako je potrebno.

Ostale ugrađene validacije

Osim obveznog polja, koje je najčešći oblik validacije, moguće je postaviti ograničenje na minimalni ili maksimalni broj znakova za neko polje te odrediti interval za minimalni ili maksimalni uneseni broj.

Također je moguće staviti više validacija na isto polje. Polje može biti obvezno i može imati maksimalni broj znakova.

Question.cs

```

namespace QuizManager.Model
{
    public class Question
    {
        [Required]
        [Range(1, 50)]
        public int Points { get; set; }

        [Required]
        [StringLength(2000, MinimumLength = 5)]
        public string QuestionText { get; set; }
    }
}

```

```
}  
}
```

Također je moguće definirati proizvoljne poruke za prikaz korisniku. Naravno, postoji i mogućnost prikaza lokalizirane poruke, no o tome će biti više riječi u kasnijim poglavljima.

Primjer prilagođene poruke:

`Question.cs`

```
namespace QuizManager.Model  
{  
    public class Question  
    {  
        [Required]  
        [Range(1, 50, ErrorMessage = "Broj bodova mora biti  
između 1 i 50.")]  
        public int Points { get; set; }  
  
        [Required]  
        [StringLength(2000, MinimumLength = 5)]  
        public string QuestionText { get; set; }  
    }  
}
```

Više o validaciji i anotacijama: <https://docs.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-6.0>

Padajući izbornik

Padajući izbornik, odnosno combobox ili dropdown list, nezaobilazna je kontrola u gotovo svakoj aplikaciji. Koristi se u svakoj situaciji gdje trebamo osigurati unos polja koje je strani ključ, odnosno 1-N veza.

Vrste dropdown kontrola

Kod implementacije dropdown kontrole treba odabrati pristup prema količini podataka i načinu korištenja.

1. Statički dropdown

Statički dropdown koristi se kada postoji mali broj opcija, primjerice manje od 20.

Primjer:

- status narudžbe
- tip korisnika
- kategorija s malim brojem vrijednosti

Kod ovog pristupa server već kod otvaranja forme šalje sve opcije, a view ih samo iscrta.

2. Autocomplete dropdown s dohvatom sa servera

Autocomplete dropdown koristi se kada postoji veći broj opcija. Korisnik počne pisati tekst, a aplikacija šalje AJAX zahtjev serveru i dohvaća samo opcije koje odgovaraju upitu.

Primjer:

- korisnici
- gradovi
- adrese
- proizvodi
- veliki šifarnici

Ovaj pristup je obavezan dio vježbe jer pokazuje rad s AJAX pozivima, custom kontrolom i server-side pretragom.

3. Hibridni dropdown

Hibridni pristup koristi se kada broj opcija nije jako velik, ali je korisniku svejedno korisno omogućiti pretragu unutar dropdowna. Sve opcije se učitaju unaprijed, ali se filtriranje radi na klijentu, bez dodatnog odlaska na server.

Autocomplete dropdown u Create i Edit formama

Kod običnog statičkog dropdowna controller za `Create` formu šalje sve opcije. Kod `Edit` forme šalje sve opcije i ID trenutno odabrane opcije.

Kod autocomplete dropdowna pristup je drugačiji.

Create forma

Kod **Create** forme nije potrebno odmah slati sve opcije. Kontrola zna endpoint koji će pozvati kada korisnik počne pisati.

Primjer:

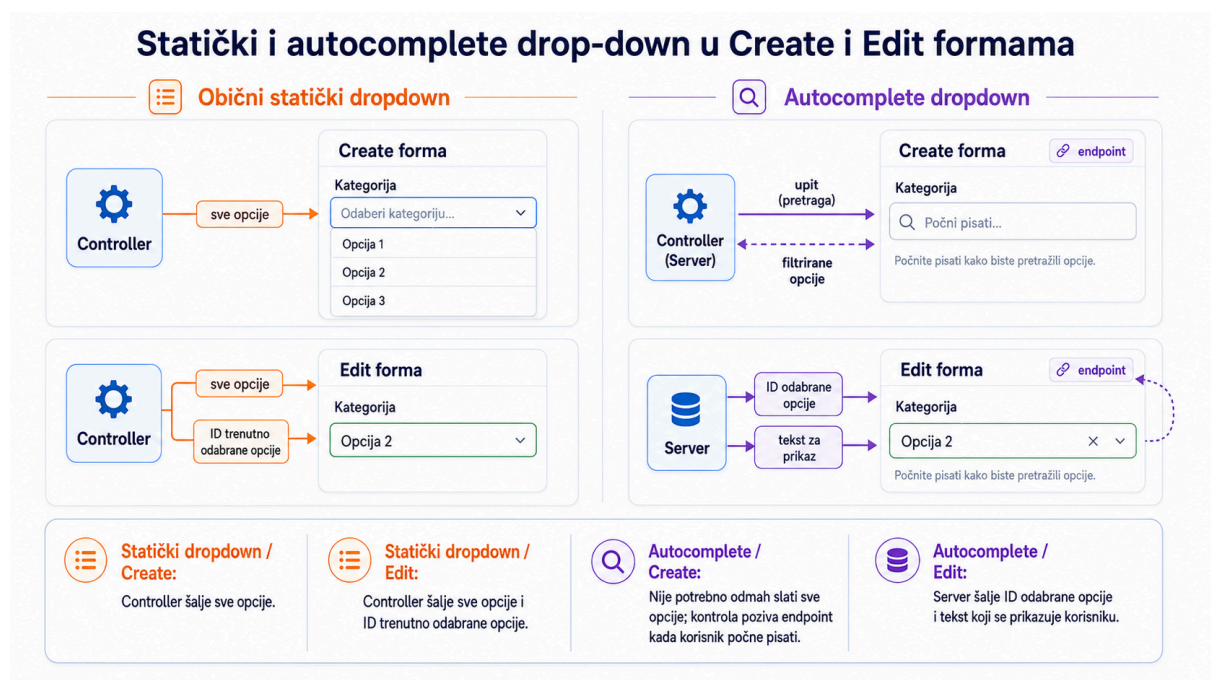
- korisnik počne pisati **Zag**
- JavaScript šalje AJAX zahtjev na endpoint za pretragu gradova
- server vraća prvih nekoliko rezultata
- korisnik odabire jedan rezultat
- u hidden input sprema se ID odabrane opcije

Edit forma

Kod **Edit** forme potrebno je prikazati već odabranu vrijednost. Zato server mora poslati:

- ID odabrane opcije
- tekst koji se prikazuje korisniku

Primjer: ako je u bazi spremljen **CityId = 5**, forma treba prikazati naziv grada, primjerice **Zagreb**, a ne samo ID.



Performanse autocomplete pretrage

Za nekoliko tisuća zapisa jednostavna pretraga u bazi najčešće nije problem. Primjerice, pretraga kroz 3.000 korisnika ili gradova može se riješiti običnim filtriranjem i vraćanjem prvih 10–20 rezultata.

Okvirno:

- do nekoliko tisuća zapisa — jednostavna pretraga je uglavnom dovoljna
- do oko 10.000 zapisa — često je i dalje prihvatljivo uz dobar upit i indeks
- iznad toga treba razmisliti o indeksima, `StartsWith` pretraži ili full-text search rješenju

Full-text search može biti brz, ali ima cijenu: dodatni indeks može biti velik i složen za održavanje. Zato se ne koristi automatski za svaki dropdown.

Primjerice, ako promotrimo klasu `Quiz`, strani ključ je kategorija i za svaki kviz potrebno je odabrati kategoriju.

`Quiz.cs`

```
namespace QuizManager.Model
{
    public class Quiz
    {
        ...
        public DateTime DateCreated { get; set; }
        public int? CategoryId { get; set; }
        ...
    }
}
```

Kako bismo prikazali padajući izbornik za odabir kategorije, potrebno je:

- Modificirati controller i u željenoj akciji proslijediti sve moguće kategorije koje se mogu izabrati.
- Modificirati view i osigurati da se za proslijeđene kategorije iscrtava padajući izbornik.

Modifikacija controllera svodi se na spremanje svih mogućih kategorija u

`ViewBag`, ili u model, ovisno o implementaciji.

`QuizController.cs` — dodavanje sadržaja za padajući izbornik

```

var selectItems = new List<System.Web.Mvc.SelectListItem>
();

// Polje je opcionalno
var listItem = new SelectListItem();
listItem.Text = "- odaberite -";
listItem.Value = "";
selectItems.Add(listItem);

foreach (var category in _db.QuizCategories)
{
    listItem = new SelectListItem();

    // Popuniti polja Text (ono što se prikazuje korisniku)
    i Value (id)
    selectItems.Add(listItem);
}

ViewBag.PossibleCategories = selectItems;

```

Ovakav identični kod potrebno je izvršiti na ovim mjestima:

- Prilikom akcije `Create`, odnosno stvaranja novog objekta.
- Prilikom akcije `[Post]Create`, odnosno procesiranja podataka s forme, u slučaju da validacija ne prođe i potrebno je ponovno prikazati isti view.
- Prilikom akcije `Edit`, odnosno uređivanja postojećeg objekta.
- Prilikom akcije `[Post]Edit`, odnosno procesiranja podataka s forme, u slučaju da validacija ne prođe i potrebno je ponovno prikazati isti view.

Iz tog razloga preporučljivo je taj kod izdvojiti u posebnu funkciju.

U viewu je potrebno dodati odgovarajući poziv HTML helper metode.

`_CreateOrEdit.cshtml`

```

<div class="form-group">
    <label class="control-label">Category</label>
    <select asp-for="CategoryID" asp-items="ViewBag.Possibl

```

```
eCategories" class="form-control"></select>
</div>
```

Ponovno korištenje dropdown kontrole

Cilj nije svaki dropdown implementirati ispočetka. Bolji pristup je napraviti jednu dobru autocomplete kontrolu, dobro je testirati i zatim je ponovno koristiti.

Preporučeni pristup:

1. Napraviti partial view za autocomplete kontrolu.
2. Definirati jasan format podataka koji kontrola očekuje.
3. Definirati endpoint koji vraća rezultate pretrage.
4. Testirati kontrolu na jednom entitetu.
5. Nakon toga istu kontrolu koristiti na drugim formama.

Kod korištenja AI alata korisno je prvo ručno ili iterativno dovesti jednu formu do dobrog stanja, a zatim AI-u dati uputu da ostale forme napravi po istom obrascu.

JavaScript

JavaScript je skriptni jezik koji se izvodi u svim web preglednicima i neophodan je za izradu atraktivnih i funkcionalnih web aplikacija. Zadnjih nekoliko godina najčešće se koristi u kombinaciji s jQuery dodatkom za efikasno i jednostavno manipuliranje podacima i elementima.

Vrlo je sličan C programskom jeziku, iako dopušta veliku razinu fleksibilnosti po pitanju tipova varijabli i samog prevođenja, što ga s druge strane čini kompliciranijim i otežava pronalazak pogrešaka.

Ako želimo da se neki JavaScript kod izvede prilikom iscrtavanja HTML stranice u internet pregledniku, tada takav kod stavljamo u poseban HTML tag `script`.

Unutar `script` elementa mogu se nalaziti i funkcije koje neće odmah biti izvedene, nego tek kad se pozovu. Također postoji JavaScript kod koji se poziva na neku akciju koju korisnik izvodi, primjerice klik gumba ili prelazak mišem preko određenog HTML elementa.

```
Home/Index.cshtml
```

```

<h3>We suggest the following:</h3>
<ol class="round">
  <li class="one">
    ...
    <a href="http://go.microsoft.com/fwlink/?LinkId=245
151">Learn more...</a>
  </li>
  <li class="two">
    ...
    <a href="http://go.microsoft.com/fwlink/?LinkId=245
153">Learn more...</a>
  </li>
  <li class="three" onclick="onLiElementClick()">
    ...
    <a href="http://go.microsoft.com/fwlink/?LinkId=245
157">Learn more...</a>
  </li>
</ol>

<script type="text/javascript">
  function onLiElementClick() {
    alert('<li> element clicked!');
  }

  alert('Kod koji se odmah izvodi!');
</script>

```

- `onclick="onLiElementClick()"` — kod koji se izvodi na klik tog elementa.
- `function onLiElementClick()` — funkcija koja se izvodi tek na poziv.
- `alert('Kod koji se odmah izvodi!')` — kod koji se izvodi pri iscrtavanju stranice.

Rukovanje HTML elementima u samom JavaScriptu prilično je nezgrapno, stoga je preporučljivo koristiti jQuery za manipulaciju elementima i podacima.

Redosljed izvođenja JS koda i organizacija view datoteka

Zbog toga što browser izvodi kod stranice odozgo prema dolje, odnosno HTML/JS kod koji se nalazi prije bit će prije izveden, nije svejedno gdje stoji

`script` tag unutar kojeg se nalazi JS kod.

Također treba uzeti u obzir način kako se definira stranica u MVC radnom okviru. Na tipičnoj stranici elementi su raspoređeni ovako:

- Header dio stranice nalazi se u `_Layout.cshtml`.
- Footer stranice nalazi se u `_Layout.cshtml`.
- Ostali sadržaj, primjerice popis klijenata, nalazi se u `Client/Index.cshtml`.

Layout pogled služi za iscrtavanje zajedničkih dijelova sučelja koji se koriste na ostalim stranicama. Najčešće su to izbornik, footer, obavijest o korištenju kolačića i slično.

Slijedi skraćeni dio `_Layout` stranice uz objašnjenje bitnijih dijelova koda:

1. Mjesto za meta podatke za aplikaciju, odnosno ono što obično ide u `head` dio. Primjerice, `title` tag se generira iz `ViewData` dictionaryja, a sadržaj `title` taga puni se u svakom viewu posebno.
2. Uključivanje CSS datoteka. Od .NET Core verzije moguće je definirati različite biblioteke ovisno o tome gdje se nalazi aplikacija, pomoću `environment` taga. Najčešće će se različite datoteke uključiti ovisno o tome radi li se o dev ili produkcijskom/testnom okruženju. Ovaj element nije obavezan.
3. Navigacija. Više informacija:
<https://getbootstrap.com/docs/5.1/components/navbar/>
4. Poziv partial viewa za prihvaćanje kolačića (`cookie consent`).
5. Ključni poziv: `RenderBody`. Ovaj dio je jedini obavezan unutar `_Layout` datoteke. Na to mjesto ubacuje se sadržaj viewa koji gledamo. Primjerice, ako je pozvana akcija `Client/Edit`, tada se unutar `_Layout` stranice u pozivu `RenderBody` iscrtava view `Views/Client/Edit.cshtml`. Analogno vrijedi za bilo koji drugi view.
6. Footer, odnosno zajednička komponenta vidljiva na svakoj stranici.
7. Uključivanje JS biblioteka. Preporučljivo ih je uključiti na kraju dokumenta kao u ovom primjeru, ali mogu se uključiti i u `head` dijelu.
8. Bitni dio: `@section`. S obzirom na to da se unutar `RenderBody` iscrtava svaki view posebno, ponekad je potrebno u odnosu na neki view ubaciti elemente u ostale dijelove stranice, primjerice u `head`, `navbar`, `footer` ili, kao u ovom slučaju, omogućiti dodavanje JavaScript koda koji će se izvršavati nakon

što su učitane sve JS biblioteke navedene iznad. Ovo je najčešće korišteni `section`, ali moguće je dodati i vlastite `section` dijelove, gdje neki mogu biti obvezni, a neki opcionalni.

`_Layout.cshtml`

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - Vjezba.Web</title>

  <environment include="Development">
    <link rel="stylesheet" href="~/css/site.css" />
  </environment>

  <environment exclude="Development">
    <link rel="stylesheet" href="~/css/site.min.css" />
  </environment>
</head>
<body>
  <nav class="navbar navbar-inverse navbar-fixed-top">
    ...
  </nav>

  <partial name="_CookieConsentPartial" />

  <div class="container body-content">
    @RenderBody()
    <hr />
    <footer>
      <p>&copy; 2019 - Vjezba.Web</p>
    </footer>
  </div>

  <environment include="Development">
    <script src="https://ajax.aspnetcdn.com/ajax/jquery
```

```

y/jquery-3.3.1.min.js"></script>
    ...
    <script src="~/js/site.js" asp-append-version="true"></script>
</environment>

<environment exclude="Development">
    <script src="https://ajax.aspnetcdn.com/ajax/jquery/jquery-3.3.1.min.js"></script>
    ...
    <script src="~/js/site.min.js" asp-append-version="true"></script>
</environment>

@RenderSection("Scripts", required: false)
</body>
</html>

```

Client/Index.cshtml

```

@model List<Client>

<h2>Popis klijenata</h2>
...

@section Scripts {
    <script type="text/javascript">
        alert('Poziv iz view-a Client/Index.cshtml');
    </script>
}

```



Važna napomena: `@section` ne funkcioniše u mehanizmu `PartialView`, zato se dodaje samo u "pravi" viewovima.



Ako view koristi jQuery, JavaScript kod koji ovisi o jQueryju treba staviti u `@section Scripts`. Taj se dio ubacuje u layout nakon što su učitane JavaScript biblioteke. Ako se skripta napiše prije učitavanja jQueryja, kod neće raditi.

CSS se u pravilu učitava na početku stranice kako bi korisnik što prije vidio pravilno stiliziran sadržaj. JavaScript se u pravilu učitava na kraju stranice jer funkcionalnost može pričekati dok se osnovni sadržaj ne prikaže.

jQuery

jQuery je biblioteka koja pruža niz funkcionalnosti za manipulaciju HTML elementima te je osnova za veliku većinu modernih JavaScript plugina.

Bazira se na nekoliko bitnijih koncepata koji se često koriste u ASP.NET MVC aplikacijama:

- jQuery selectors — mehanizam za određivanje na koje elemente se primjenjuje željena manipulacija.
- jQuery AJAX — mehanizmi za izvođenje jednostavnih AJAX poziva.
- jQuery events — mogućnost pridruživanja koda uz događaje nad specificiranim HTML elementima.
- jQuery data — mehanizam vezivanja podataka uz HTML elemente.

Budući da u pravilu želimo izvršiti kod tek kad je cijeli HTML učitao, koristi se posebna jQuery sintaksa za izvršavanje koda u trenutku kad je DOM, odnosno JavaScript pozadinski objekt vezan uz HTML stranicu, učitao.

```
<script type="text/javascript">
    $(document).ready(function () {
        // Ovdje ide kod koji se treba
        // izvršiti u trenutku kad je DOM
        // učitao.
    });
</script>
```

Ili kraće:


```

<script type="text/javascript">
    $(function () {
        // Ovdje ide kod koji se treba
        // izvršiti u trenutku kad je DOM
        // učitano.
    });
</script>

```

Većina koda koji se izvršava radi se ili kad je cijeli DOM učitano ili na akciju korisnika.

jQuery selectors

Primjer 1: Nakon što se DOM učita, postaviti pozadinsku boju tablice s `id="tbl-clients"` u tabličnom prikazu klijenata na žuto.

`Client/Index.cshtml`

```

@model List<Client>

<h2>Pregled klijenata</h2>

<table id="tbl-clients" class="table table-condensed">
    <thead>
        ...
    </thead>
    <tbody>
        ...
    </tbody>
</table>

@section scripts {
    <script type="text/javascript">
        $(function () {
            $("#tbl-clients").css("background", "yellow");
        });
    </script>
}

```

Kada označavamo specifični element po njegovom `id` svojstvu, koristimo `#`.

Primjer 2: Klikom na pojedini redak (`tr`) želimo da se tom retku svi `td` elementi popune tekстом `-`.

Client/Index.cshtml

```
<table id="tbl-clients" class="table table-condensed">
  <thead>
    ...
  </thead>
  <tbody>
    @foreach (var client in Model)
    {
      <tr onclick="changeText(this)">
        ...
      </tr>
    }
  </tbody>
</table>

@section scripts {
  <script type="text/javascript">
    ...
    function changeText(tr) {
      $(tr).find("td").text("-");
    }
  </script>
}
```

Sve skripte koje želimo da se izvode staviti ćemo u `@section scripts`. Treba imati na umu da `@section scripts` nije dostupan u partial viewovima.

Primjer 3: Prilikom prelaska mišem preko pojedinog retka želimo taj redak duplicirati i ubaciti na kraj tablice.

Client/Index.cshtml

```
<table id="tbl-clients" class="table table-condensed">
  ...
  <tbody>
    @foreach (var client in Model)
```

```

    {
        <tr onclick="changeText(this)" onmouseover="duplicateRow(this)">
            ...
        </tr>
    }
</tbody>
</table>

@section scripts {
    <script type="text/javascript">
        ...
        function duplicateRow(tr) {
            var dupl = $(tr).clone();
            $("table.table tbody").append(dupl);
        }
    </script>
}

```

jQuery selectors dokumentacija: <http://api.jquery.com/category/selectors/>

Datumska kontrola – Datepicker

Pri izradi web aplikacija često se pojavljuje problem s rukovanjem datumskim podacima. Najčešći problem je format datuma, koji je različit ovisno o jeziku koji korisnik koristi.

U .NET-u, kad govorimo o jeziku, primarno mislimo na `CultureInfo` objekt, koji se veže uz neku specifičnu državu ili područje. Primjerice, postoji `CultureInfo` objekt za Englesku (`en-GB`), SAD (`en-US`), Hrvatsku (`hr-HR`) i slično. Kad radimo s datumima, u pravilu je dovoljno koristiti samo dva slova (`hr` ili `en`) jer takav način primjereno definira format datuma.

Gdje nastaje problem? Problem nastaje ako korisnik u postavkama preglednika ima postavljen prioritetni jezik, primjerice `hr`, dok se na serveru očekuje `en`. Parsiranje datuma tada će biti neuspješno jer US format datuma glasi `MM/dd/yyyy`, dok je na našim područjima uobičajeno `dd.MM.yyyy`.

Dodatni problem pojavljuje se kod definiranja formata datuma na klijentskim komponentama. Format datuma definiran u C# jeziku kao `dd.MM.yyyy` odgovara

klijentskoj inačici `dd.mm.yyyy`, odnosno razlikuje se veliko i malo slovo za mjesece.

U sklopu ovog kolegija neće se ulaziti u probleme same višezječnosti i prilagodbe aplikacije za više jezika, ali će se ukazati na potrebne korake u rješavanju problema formata datuma ovisno o klijentskim postavkama.

Klijentski preglednik može imati postavljeno nekoliko jezika koje klijent koristi, često prioritarnim redoslijedom. U Google Chrome pregledniku jezik se definira u postavkama (`Settings`).

S obzirom na postavke jezika, prilikom svakog zahtjeva za web stranicu automatski se ta informacija šalje zajedno sa zahtjevom kako bi poslužiteljska aplikacija mogla odlučiti koji jezik prikazati korisniku.

ASP.NET MVC na .NET 6.0 radnom okviru nudi opciju automatskog prepoznavanja klijentskog jezika upravo po ovom parametru postavljenom u pregledniku.

`Program.cs`

```
...
var supportedCultures = new[]
{
    new CultureInfo("hr"),
    new CultureInfo("en-US")
};

app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture("hr"),
    SupportedCultures = supportedCultures,
    SupportedUICultures = supportedCultures
});

app.MapControllerRoute(...)
```

Gornjim odsječkom koda definirano je sljedeće:

- Jedini jezici koji su podržani u našoj aplikaciji su hrvatski i engleski.
- Ako nije drukčije definirano ili se zahtijeva neki drugi jezik, pretpostavit ćemo hrvatski jezik.



Važno: Poziv `app.UseRequestLocalization()` obvezno je napraviti prije poziva `app.UseEndpoints()`.

Nakon što je u `Startup` klasi definirana višjezičnost na gornji način, automatski se postavlja ispravi jezik za svaki request i dostupan je bilo gdje u aplikaciji pozivom:

```
System.Globalization.CultureInfo.CurrentCulture  
System.Globalization.CultureInfo.CurrentUICulture
```

`Culture` objekt sadrži niz informacija o tome kakav je format datuma, decimalnih brojeva i slično, a detalje je moguće pregledati kroz debug način rada.

Odabir date picker kontrole

Za unos datuma postoje tri česta pristupa:

1. native HTML kontrola (`input type="date"`)
2. JavaScript plugin
3. custom kontrola generirana ili prilagođena za aplikaciju

Native HTML kontrola je jednostavna, ali često nije dovoljno fleksibilna za ozbiljnije aplikacije. JavaScript pluginovi mogu biti korisni, ali treba paziti jesu li održavani i kompatibilni s verzijom frameworka i UI biblioteke koja se koristi.

Danas je realna opcija napraviti vlastitu jednostavnu kontrolu uz pomoć AI alata. Prednost je što se može prilagoditi točno potrebama aplikacije, ali ju je potrebno dobro testirati.

Posebno treba provjeriti:

- format datuma
- rad s hrvatskom i engleskom kulturom
- validaciju praznih i neispravnih vrijednosti
- ponašanje kod edit forme
- spremanje vrijednosti na server

Preporučeni način rada s AI alatima

AI alati su najkorisniji kada trebaju ponoviti postojeći kvalitetan obrazac. Zato je bolje prvo napraviti jednu formu ili jednu kontrolu kvalitetno, testirati je i tek onda tražiti od AI-a da isti pristup primijeni na ostale entitete.

Loš pristup:

- generirati 10 stranica odjednom
- dobiti 10 približno sličnih rješenja
- kasnije otkriti da većina ne radi ispravno

Bolji pristup:

1. Napraviti jednu formu do kraja.
2. Provjeriti validaciju, dropdown, edit, delete i testove.
3. Popraviti stil i ponašanje.
4. Tek tada tražiti od AI-a da napravi ostale forme po istom obrascu.

AI je posebno dobar u repliciranju postojećeg rješenja u drugi kontekst. Zato je vrijedno uložiti više vremena u prvi primjer.